Enrichment and Imputation in Spark

In this train, we'll explore methods to enrich datasets and handle missing data through imputation using Spark. These methods are essential for ensuring high-quality, trustworthy datasets in data engineering workflows.

Learning objectives

By the end of this train you should be able to:

- [Placeholder for objective 1]
- [Placeholder for objective 2]
- [Placeholder for objective 3]

Learning Objectives

- Perform data enrichment by adding or transforming features using Spark functions like `withColumn()` and `join()`.
- Handle missing data using methods like `dropna()`, `fillna()`, and conditional logic via `when()` and `otherwise()`.
- Understand how to inspect, clean, and prepare structured datasets using Spark's DataFrame API.

Data Enrichment Techniques in Spark

Data enrichment improves the value of your data by either adding new fields or enhancing existing ones.

💡 Example 1: Adding a Derived Column

```
from pyspark.sql.functions import col

# Add a new column "is_humid" based on relative humidity
df = df.withColumn("is_humid", col("rhum") > 80)
df.show()
```

Explanation: This creates a boolean column `is_humid` that is `True` if relative humidity (`rhum`) is greater than 80. This is a form of contextual enrichment useful in climate-based analytics.

💡 Example 2: Enriching Via Join

```
# Join station metadata to weather data
enriched_df = df.join(metadata_df, on="station", how="left")
```

Explanation: This left join adds metadata such as station name or location details to the main dataset, enriching it with external contextual information.

Handling Missing Data Through Imputation in Spark

Missing data is a common issue that must be addressed to avoid skewed analytics or processing errors.

💡 Example 3: Dropping Nulls

```
# Drop rows where any column is null
df_clean = df.dropna()
```

Explanation: Removes all rows that contain `NULL` values in any column. Use with caution—this can result in major data loss if missing values are widespread.

💡 Example 4: Filling Nulls

```
# Replace NULL values in specific columns
df = df.fillna({'rhum': 50, 'temp': 20.0})
```

Explanation: Replaces missing values in the `rhum` column with 50 and `temp` with 20.0. This method is best used when there's a known reasonable default value.

💡 Example 5: Conditional Imputation With Logic

```
from pyspark.sql.functions import when

# Replace NULLs in 'clamt' conditionally
df = df.withColumn("clamt", when(col("clamt").isNull(),
0).otherwise(col("clamt")))
```

Explanation: This uses a conditional expression to replace `NULL` values in the `clamt` column with 0 while keeping all other values intact. This gives greater control than `fillna()`.

Conclusion

Data enrichment and imputation are indispensable in Spark-based data pipelines. These transformations not only improve the usability and trustworthiness of data but also ensure efficient downstream processing. Spark's expressive DataFrame API makes it possible to handle missing values and enrich datasets in a scalable and readable manner.

▼Key Takeaways ✖

- **Takeaway 1:** Enrichment adds contextual or derived data to enhance dataset usability.
- **Takeaway 2:** Missing data should be handled thoughtfully using `dropna()` or `fillna()`.
- **Takeaway 3:** Use `when()` and `otherwise()` for conditional data correction or imputation.
- **Takeaway 4:** Schema planning and inspection is critical before applying transformations.
- **Takeaway 5:** Efficient Spark operations improve quality without inflating processing costs.